CISC 3130 - Homework 4
Name: Changmin Lee
Big-O Table and Data Structure Choices

Part A: Big-O Table

Method	Structures	Big-O	Why
addSong	List, HashMap	O(1) avg	Add at tail and save ID.
findSong	HashMap	O(1) avg	Find song by its ID.
play	Map, stack, counts	O(1) avg	Look up song, save old one, update count.
queueLater	Map, deque	O(1) avg	Check ID and add to back.
queueNext	Map, deque	O(1) avg	Check ID and add to front.
next	Deque, map, list	O(q+1) avg	May skip removed songs in queue.
back	Stack, map, deque	O(h+1) avg	May skip removed songs in history.
removeSong	List, maps	O(1) avg	Find node by ID and fix the links.
showPlaylist	Linked list	O(n)	Go through each node once.

n = number of playlist songs, q = Up Next entries, and h = history entries. HashMap is O(1) on average.

Part B: Why I Chose These Structures

1. Playlist - Doubly Linked List
I used a doubly linked list because I want the songs to stay in order. Each node knows the previous and next song. I also put each node in a HashMap using the song ID, so I can find and remove it without searching the whole list. If I used a regular array, deleting a song in the middle would take O(n) because other songs need to move. I wrote the nodes myself instead of using Java LinkedList.

2. Up Next - ArrayDeque
I used ArrayDeque for Up Next because songs can be added to the front or the back. The addFirst() and addLast() methods are O(1) amortized. A regular array would need O(n) to insert at the front because the elements need to move. I reused ArrayDeque instead of building my own deque because it was easier to work with.